



RTL Design Sherpa

STREAM Characterization Nexys A7 Project Guide 0.90

July 2026

Table of Contents

1 Document Information.....	7
1.1 References.....	7
1.2 Terminology.....	7
1.3 Revision History.....	8
2 Overview.....	8
2.1 What it characterizes.....	8
2.2 The flow matrix.....	9
2.3 Board and tools.....	9
2.4 Status.....	10
3 How It Works.....	10
3.1 The spine (primary flow).....	10
3.1.1 Figure 2.1: STREAM Characterization Harness Spine.....	10
3.2 Bridge slaves and base addresses.....	10
3.3 Key RTL blocks.....	11
3.4 The instrumentation harness.....	12
4 Build and Run.....	12
4.1 Make targets.....	12
4.2 Host tools (flows-stream-bridge/host/).....	13
4.3 Port autodetect and identity.....	14
5 Harness CSR Configuration.....	14
5.1 Address map and by-name access.....	14
5.1.1 Waveform 4.1: Descriptor Load and Kick.....	15

5.2 harness_csr — control / status (base 0x0001_0000).....	15
5.3 Timer and the latency knob.....	16
5.4 CRC, kick, and observability.....	17
5.5 STREAM monitor window (0x1000).....	18
5.6 Descriptor format (desc_ram @ 0x0002_0000).....	18
6 Simulation / Silicon Equivalence.....	18
6.1 The testbench.....	18
6.2 Test levels.....	19
6.3 Sibling flows.....	19
7 Troubleshooting.....	19
7.1 Current state.....	19
7.2 Known issues.....	20
7.3 Cannot find the board.....	20
7.4 Not simulatable.....	20
7.5 Board fit vs sim geometry.....	20
7.6 Which README to trust.....	21

List of Figures

Figure 2.1: STREAM Characterization Harness Spine.....	10
--	----

List of Tables

Table 1: Reference documents.....	7
Table 2: Revision history.....	8
Table 3: Characterization axes and reports.....	8
Table 4: The flow matrix.....	9
Table 5: Board and tool requirements.....	9
Table 6: Bridge slave address map.....	10
Table 7: Key RTL blocks.....	11
Table 8: flows-stream-bridge make targets.....	13
Table 9: harness_csr control / status.....	15
Table 10: Timer and memory-latency registers.....	16
Table 11: CRC, kick, and observability registers.....	17
Table 12: cocotb test levels.....	19
Table 13: Known issues and workarounds.....	20

List of Waveforms

Waveform 4.1: Descriptor Load and Kick.....	15
---	----

1 Document Information

This is the operator / developer guide for the Nexys A7 **STREAM Characterization** project (projects/NexysA7/stream_characterization). It explains what the project characterizes, how the harness works, how to build / simulate / program / run it, and how the harness CSR is configured.

The device under test is the in-house **STREAM scatter-gather DMA** (stream_top_ch8, from projects/components/dmas/stream/). This guide documents how the harness drives and measures it — not the DMA core's internals. The STREAM core's own architecture specifications (HAS/MAS) live under projects/components/dmas/stream/docs/.

1.1 References

Reference documents

Source	Title
RTL Design Sherpa	README.md, PORT_MAP.md, DMA_UTILIZATION_MEASUREMENT.md
RTL Design Sherpa	flows-stream-bridge/host/ ADDRESS_MAP.md (bridge + CSR map)
RTL Design Sherpa	reports/ {perf,compression,area,ext_addressing}/README.md
RTL Design Sherpa	bin/TBClasses/harness/ (shared UART-char collateral)
ARM	AMBA AXI4 / AXI4-Lite / APB Protocol Specifications

1.2 Terminology

Flow

One cell of the characterization matrix — a {DMA} × {bridge} pairing built as a flows-<dma>-<bridge>/ directory.

Harness CSR

The instrumentation register block (harness_csr) that arms the pattern generators, latency model, bus meters, and CRC checkers around the DUT.

MonBus

STREAM's internal 64-bit monitoring bus; its packets are drained to debug_sram (bulk trace) and stream_err (IRQ FIFO), and optionally compressed.

Observer

axi4_dma_observer — non-perturbing valid/ready snoops on the DMA payload buses that feed the utilization buckets and the compression counters.

1.3 Revision History

Revision history

Version	Date	Notes
0.90	2026-07-18	Initial project guide

2 Overview

2.1 What it characterizes

This project measures the in-house **STREAM scatter-gather DMA** on a Digilent Nexys A7-100T (Artix-7 xc7a100tcsg324-1, 100 MHz), driven entirely over one USB-UART link, across four axes:

Characterization axes and reports

Axis	Report	Headline
Performance / utilization	reports/perf/	100% datapath utilization across the 40-config matrix (~1525 MB/s, 128-bit datapath)
MonBus trace compression	reports/compression/	63.0% (64-bit codec), 75.7% (half-beat 32-bit) reduction
Area	reports/area/	bare DMA OOC vs an open-source iDMA comparison

Axis	Report	Headline
Extended addressing	reports/ ext_addressing/	row/row $\approx$ 1.5 GB/s vs col/col $\approx$ 0.26–0.32 GB/s

2.2 The flow matrix

The project is framed as a 2×2 comparison — {STREAM in-house, Vivado MCDMA} × {in-house bridge, Vivado bridge} — so every cell is measured through the same shared instrumentation harness:

The flow matrix

Flow	DUT	Status
flows-stream-bridge/	STREAM (stream_top_ch8) + in-house bridge	Primary, most complete — cocotb gate + full FPGA sweeps
flows-vivado-mcdma/	Vivado AXI- MCDMA IP + in- house bridge	Skeleton (blinks LED; FPGA-only — VHDL IP can't Verilate)
flows-idma-bridge/	PULP iDMA	Area + datapath-perf cosim only (feeds the iDMA comparison)
flows-stream-vivado- bridge/, flows- vivado-mcdma-vivado- bridge/	future cells	Listed in make help; not built

This guide focuses on the primary flows-stream-bridge flow; the others are inventoried in Chapter 6.

2.3 Board and tools

Board and tool requirements

Item	Detail
Board	Digilent Nexys A7-100T (xc7a100tcsg324-1), 100 MHz
Build	Vivado 2025.1
Sim	Verilator + cocotb / cocotb_test / pytest
Host	Python 3 + pyserial over UART

Item	Detail
	(115200 8N1)

2.4 Status

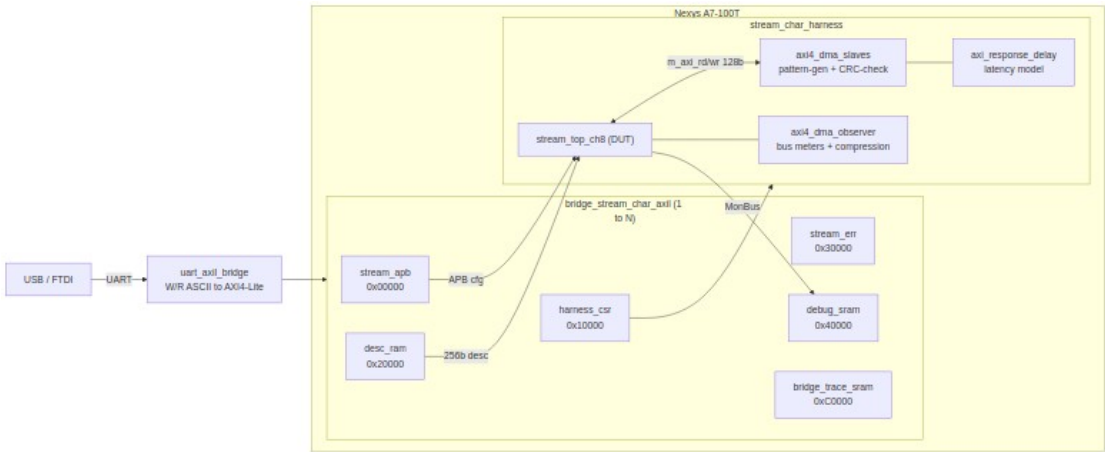
The primary stream-bridge flow is working — cocotb gate + full FPGA sweeps are collected, with data in the perf (v1.4) and compression (v1.3) writeups (progress/STATUS = 90-DONE). flows-vivado-mcdma is a skeleton; flows-idma-bridge is area + cosim only.

3 How It Works

3.1 The spine (primary flow)

The host drives one UART link into a generated 1→N AXI4-Lite/APB fabric that fans out to the DUT config, the harness CSR, and the trace/error memories:

3.1.1 Figure 2.1: STREAM Characterization Harness Spine



STREAM characterization harness spine

Source: [01_harness_spine.mmd](#)

3.2 Bridge slaves and base addresses

From host/ADDRESS_MAP.md and PORT_MAP.md (the current design; the older flows-stream-bridge/README.md describes a superseded 5-slave decoder — trust the address-map docs):

Bridge slave address map

Base	Size	Slave	Width	Protocol
0x0000_000	4 KB	stream_apb	32b	APB

Base	Size	Slave	Width	Protocol
0		(STREAM config)		
0x0001_000 0	4 KB	harness_csr	32b	AXIL
0x0002_000 0	64 KB	desc_ram (32 B/desc)	256b	AXIL
0x0003_000 0	4 KB	stream_err (MonBus IRQ-FIFO drain)	64b	AXIL
0x0004_000 0	256 KB	debug_sram (MonBus bulk trace)	64b	AXIL
0x0008_000 0	4 KB	dma_axil (tied off, reserved MCDMA)	32b	AXIL
0x000c_000 0	64 KB	bridge_trace_sram (bridge MonBus trace)	64b	AXIL

Every MonBus-carrying AXI path is 64-bit. The descriptor read path is a direct 256-bit AXI4 (m_axi_desc ↔ desc_ram, bypassing the bridge); the DMA payload masters m_axi_rd / m_axi_wr are 128-bit into the pattern-gen / CRC-check slaves.

3.3 Key RTL blocks

Key RTL blocks

File	Role
flows-stream-bridge/rtl/stream_char_top.sv	FPGA pin-level top: instantiates the harness, led_status_driver, seven_seg_4digit.
flows-stream-bridge/rtl/stream_char_harness.sv	The integration hub: uart_axil_bridge + generated bridge_stream_char_axil + harness_csr + desc_ram + debug_sram + axi_response_delay × 2 +

File	Role
stream_char_framework/rtl/	axi4_dma_slaves + axi4_dma_observer + the DUT stream_top_ch8. Shared framework RTL: harness_csr.sv (authoritative CSR), desc_ram.sv, debug_sram.sv, axi_response_delay.sv, led_status_driver.sv, seven_seg_4digit.sv, sram_chan_tracker.sv.

3.4 The instrumentation harness

Around the DUT, the harness supplies stimulus, a memory-latency model, and non-perturbing observability:

- **axi4_dma_slaves** — an LFSR read-source pattern generator (feeds `m_axi_rd`) and a write-sink CRC checker (verifies `m_axi_wr`).
- **axi_response_delay** ×2 — a programmable memory-latency model on the read and write paths (the `RESP_DELAY` CSR, Chapter 4).
- **axi4_dma_observer** — non-perturbing valid/ready bus meters plus a burst histogram and the MonBus-compression observer, all read back through the harness CSR.

The DUT's own MonBus monitors feed `debug_sram`; the monbus compressor / half-beat packer (`USE_MON_COMPRESSION`) drives the compression characterization.

4 Build and Run

Every Makefile requires `REPO_ROOT`, so source the environment first:

```
cd /path/to/RTLDesignSherpa && source env_python
cd projects/NexysA7/stream_characterization
```

4.1 Make targets

The **top-level Makefile** is a per-flow factory — targets take a flow suffix:

```
make bitstream-stream-bridge    # build the primary flow
make program-stream-bridge
make sim-stream-bridge
```

```
make all-bitstreams          # every built flow
make docs                   # render the reports
make help                   # full 2x2 matrix + targets
```

The **primary flow** (flows-stream-bridge/Makefile) has the detailed targets:

flows-stream-bridge make targets

Target	What it does
make bitstream	regen-bridges + verify-sim + full P&R (~10–30 min) → bitstream/stream_char.bit
make program	JTAG flash
make sim	cocotb regression (dv/tests → run-all-full-parallel)
make area	out-of-context bare-stream_top_ch8 area report
make synth/make utilization/make timing	synth-only + reports
make regen-bridges/make regen-regs-nomon	regenerate the bridge / register collateral
make verify-sim	pre-bitstream gate (-k "ping or csr_read"); bypass with BITSTREAM_SKIP_VERIFY=1

The dv/tests/Makefile runs the cocotb suite by level: run-all-{gate,func,full}[-parallel][-wave] (TEST_LEVEL, 48-worker xdist, --reruns 3).

```
cd flows-stream-bridge
make bitstream
make program
make sim
```

4.2 Host tools (flows-stream-bridge/host/)

All default --port auto (autodetect) and --baud 115200.

run_characterization.py — the main sweep runner (40-config matrix: {1,2,4,8,16} desc/ch × 1–8 ch × 1 MB):

```
./run_characterization.py --port /dev/ttyUSB1 --channels 1 2 4 8 --
size 1MB --csv fpga_suite.csv
./run_characterization.py --resp-delays 0,128 --compression --mon-
config debug-compl
```

--configs, --phase, --resp-delays[-wr], --rd-prefetch, -o out.json, --dry-run; env XFER_BEATS=b sets burst size.

characterize.py — a lighter one-row-per-config sweep (cycles + MB/s).
stream_ext_suite.py / **stream_ext_char.py** — extended row/col addressing (needs the DUT built with `USE_ROW_COL_MAJOR_ADDRESSING=1`). **per_source_capture.py** — per-source MonBus isolation for compression datasets. **dump_status.py** — quick sanity (prints `BUILD_ID` `0x53545243`). **dump_monbus_sram.py** — dump the trace ring (`--base 0x00040000` STREAM, `0x000c0000` bridge). By-name access helpers: `stream_addrs.A()` (STREAM APB), `harness_addrs.H()` (harness CSR), `stream_device.Stream`.

Exact reproduce commands live in the `reports/perf/README.md` and `reports/compression/README.md` appendices.

4.3 Port autodetect and identity

`--port auto` probes each `/dev/ttyUSB*` by writing the magic `0xC0FFEE5A` to the harness SCRATCH CSR and reading it back; the board that echoes is the harness. Confirm it is alive with `dump_status.py` — the `BUILD_ID` at harness CSR `0x24` reads `0x5354_5243` (“STRC”).

5 Harness CSR Configuration

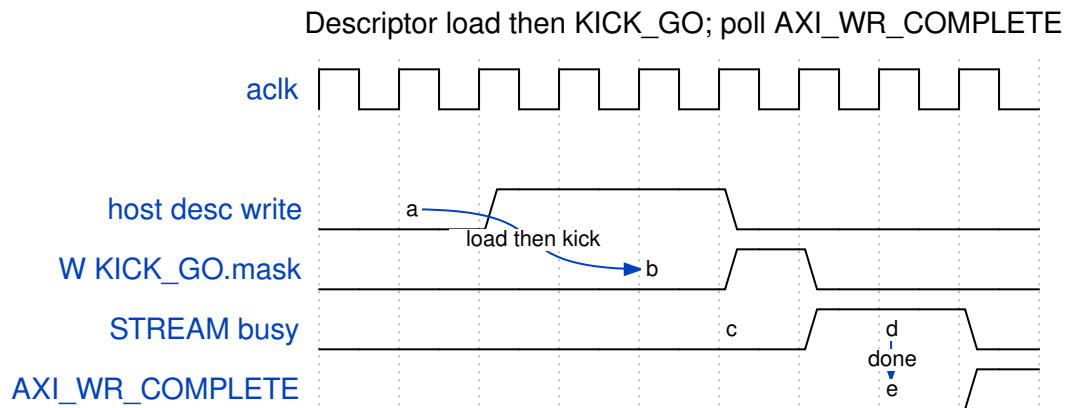
5.1 Address map and by-name access

The bridge fans out to the slaves in Chapter 2. This chapter covers the `harness_csr` slave at `HARNESS_CSR_BASE = 0x0001_0000`.

No PeakRDL RDL exists for the harness CSR. `harness_csr.sv` (`stream_char_framework/rtl/`) is authoritative; a mirror table in `stream_char_framework/bin/gen_harness_regmap.py` generates `stream_char_framework/rtl/harness_csr_regmap.py`. The host accesses it **by name** — `host/harness_addrs.py H("NAME")` resolves `HARNESS_CSR_BASE + offset` via `RegisterMap`; never hardcode offsets.

The **STREAM DUT’s** APB config, by contrast, is PeakRDL-generated (`stream_regmap.py` from `stream_regs.rdl`), accessed by name via `host/stream_addrs.py A("NAME")` — see the monitor-window note below.

5.1.1 Waveform 4.1: Descriptor Load and Kick



Descriptor load and kick

Source: [01_desc_kick.json](#)

5.2 harness_csr — control / status (base 0x0001_0000)

harness_csr control / status

Register	Offset	Field	Bits	Access	Description
CTRL	0x00	start	[0]	W	Pulse: start a measured run
CTRL	0x00	clear_s tats	[1]	W	Pulse: clear meters + sticky errors
CTRL	0x00	freeze_ trace	[2]	RW	Latch: freeze counters
CTRL	0x00	soft_re set	[3]	W	Pulse: soft reset
CTRL	0x00	cam_cle ar	[4]	W	Pulse: clear the error CAM
STATUS	0x04	stream_ irq	[0]	R	STREAM interrupt
STATUS	0x04	any_err or	[1]	R	Sticky error OR
STATUS	0x04	trace_o verflow	[2]	R	Trace ring wrapped
STATUS	0x04	clear_b	[3]	R	Clear in progress

Register	Offset	Field	Bits	Access	Description
DBG_WR_PTR	0x08	usy value	[31:0]	R	Trace write pointer (words)
DBG_OVERFLOW	0x0C	value	[0]	R	Sticky trace overflow
CRC_RD_EXPECTED / CRC_WR_EXPECTED / CRC_WR_COMPUTED	0x10 / 0x14 / 0x18	value	[31:0]	R	Aggregate CRCs
CRC_MATCH	0x1C	match / valid	[0] / [1]	R	CRC match
SCRATCH	0x20	value	[31:0]	RW	Ping / autodetect (magic 0xC0FFEE5A)
BUILD_ID	0x24	value	[31:0]	R	0x5354_5243 (“STRC”)

5.3 Timer and the latency knob

Timer and memory-latency registers

Register	Offset	Field	Bits	Access	Description
TIMER_CTRL	0x28	clear	[0]	W	Pulse: clear the cycle counter
TIMER_STATU S	0x2C	done / running / pass	[0] / [1] / [2]	R	Timer state
TIMER_CYCLE S_LO/HI	0x30 / 0x34	value	[31:0]	R	64-bit free-running cycles
TIMER_EXPECTED_BEATS	0x38	value	[31:0]	RW	Sink beat count gating “done”
RESP_DELAY	0x3C	rd_delay	[15:0]	RW	Read memory latency (aclk cycles)

Register	Offset	Field	Bits	Access	Description
RESP_DELAY	0x3C	wr_delay	[31:16]	RW	Write memory latency
TIMER_{R,W}_{FIRST,LAST}_{LO,HI}	0x40–0x5C	value	[31:0]	R	First/last R/W beat timestamps

5.4 CRC, kick, and observability

CRC, kick, and observability registers

Register	Offset	Field	Bits	Description
CRC_RD_PER_CH0.. 7	0x60–0x7C	value	[31:0]	Per-channel read CRC
CRC_WR_PER_CH0.. 7	0x80–0x9C	value	[31:0]	Per-channel write CRC
CRC_VALID_MASK / CRC_MATCH_MASK	0xA0 / 0xA4	value	[7:0]	Per-channel masks
CH0.. 3_KICK_ADDR	0xB0–0xBC	value	[31:0]	Kick shadow addr (ch 0–3)
KICK_GO	0xC0	mask	[7:0]	W: pulse a HW kick per set bit
CH4.. 7_KICK_ADDR	0xC4–0xD0	value	[31:0]	Kick shadow (ch 4–7) — split around KICK_GO
DESC_*_HS	0xD4–0xFC	value	[31:0]	Descriptor SRAM / AXI handshake counters
OBS_RD_* / OBS_WR_*	0x100–0x11C	value	[31:0]	DMA observer aggregate buckets
OBS_HIST_SEL/ DATA/TOTAL	0x120–0x128	value	[31:0]	Burst-count histogram
COMP_TIER1_A/ B/C, COMP_TIER0, COMP_*_OVF	0x1E0– 0x1FC	value	[31:0]	MonBus-compression observer readback

The kick-address block is **split around KICK_GO** (ch 0–3 below, ch 4–7 above), so a naive `BASE + 4*ch` mislands channel 4 — use the host's `kick_addr_csr(ch)` helper.

5.5 STREAM monitor window (0x1000)

STREAM's PeakRDL regmap places the **AXI monitor register block at APB offset 0x1000+** (`stream_mon_regs.rdl @ 0x1000`). The perf-counter readbacks (`RDMON_PERF_*`, `WRMON_PERF_*`, `DAXMON_PERF_*`) live there and are read **by name** via `stream_addrs.A()` — hardcoding this window broke a path once, so the by-name access is mandatory. Monitor presets (`perf-mon`, `debug-basic`, `debug-compl`, `debug-all`, `debug-core`) are built by `host/mon_configs.py`.

5.6 Descriptor format (`desc_ram @ 0x0002_0000`)

32 B / 256-bit per descriptor, 8×32 -bit words: `src_addr lo/hi (w0/1)`, `dst_addr lo/hi (w2/3)`, `length in beats (w4)`, `next_ptr (w5, 0 = end)`, `ctrl (w6: valid[0], interrupt[1], last[2], error[3], channel_id[7:4], priority[15:8], stamp[31:16])`, `reserved (w7)`. Built by `host/descriptor_builder.py`.

6 Simulation / Silicon Equivalence

The primary `flows-stream-bridge` flow has a real **UART-equivalence** cocotb sim: the testbench drives the *same* `uart_axil_bridge` RTL over the *same* ASCII W/R byte stream the FPGA host uses, and the extended-addressing tests run the *same* Python host programs. Only the bottom transport differs.

6.1 The testbench

- **TB:** `flows-stream-bridge/dv/tbclasses/stream_char_tb.py` (`StreamCharTB`). Drives `i_uart_rx / o_uart_tx` via the `CocoTBFramework` `UARTMaster / UARTMonitor` BFM's; `UART_BAUD` is raised (`CLKS_PER_BIT = 8`) for sim speed — bit-timing only, the byte stream is unchanged. Registers are addressed by name (`stream_addrs.A`, `harness_addrs.H`).
- **Backend model:** the real harness RTL, elaborated in Verilator — `axi4_dma_slaves` (LFSR pattern-gen source + CRC-check sink), `axi_response_delay`, `axi4_dma_observer`. Descriptors are built by the shared `host/descriptor_builder.py`.
- **Running the actual host program in sim:** the ext-addressing tests execute `stream_ext_suite.run_suite / stream_ext_char` under `await cocotb.external(program)()`, with a `_Bridge` shim wrapping `cocotb.function(uart_write / uart_read)` — the shared harness methodology (a `cocotb.function` transport, **not** a free-running pump). This is the same-source silicon-equivalence path.

6.2 Test levels

`dv/tests/test_stream_char.py` runs by `TEST_LEVEL`:

cocotb test levels

Level	Cocotb tests
gate	ping
func	+ desc_load, csr_read, apb_config, desc_perf, rw_perf, obs_equiv, dma_1ch, dma_2ch
full	+ dma_3ch ... dma_Nch, compress_char

Plus `test_stream_char_ext_suite/test_stream_char_ext_char` (the run-the-host-program tests) and host-side unit tests (`host/test_harness_regmap.py`, `test_stream_device.py`, `test_mon_configs.py`, ...). Sim params (`BASE_RTL_PARAMS`): `NUM_CHANNELS = 4` (down from 8 for Artix BRAM), `DATA_WIDTH = 128`, `DESC_RAM_ENTRIES = 128`, `DEBUG_SRAM_WORDS = 4096`, `USE_MON_COMPRESSION = 1`. `make bitstream` runs a `verify-sim gate (-k "ping or csr_read")` so a broken elaboration is caught before a long build.

6.3 Sibling flows

- `flows-idma-bridge` has a cocotb **cosim** (`make perf / make desc-overhead`) but not the UART harness.
- `flows-vivado-mcdma` has **no** cocotb — the MCDMA IP is VHDL and Verilator cannot simulate it, so that flow is FPGA-only (rationale in its `dv/README.md`).

The shared collateral in `bin/TBClasses/harness/` (`byte_channel.py`, `uart_register_map.py`, `cocotb_axil_bridge.py` with `make_uart_channel`, `device.py`, `harness.py`) is the basis this TB builds on.

7 Troubleshooting

7.1 Current state

The primary stream-bridge flow is working: cocotb gate + full FPGA sweeps are collected, with data in the perf (v1.4) and compression (v1.3) writeups (`flows-stream-bridge/progress/STATUS = 90-DONE`). `flows-vivado-mcdma` is a skeleton (blinks an LED, drives no transfers); `flows-idma-bridge` is area + cosim only.

7.2 Known issues

Known issues and workarounds

Symptom	Cause / workaround
Configs hang after a long mixed session	State-accumulation wedge (4–7 active channels under debug-compl): AXI_RD/WR_COMPLETE = 0, and a per-run soft/cluster reset does not clear it — only make program (full reprogram) does. Reset-completeness gap, not data corruption. Tracked in rtl/amba/KNOWN_ISSUES/axi_monitor_blockready_hang_partial_channels.md.
A later no-delay run silently under-performs	Sticky RESP_DELAY: a --resp-delays sweep leaves the CSR set. Re-program before a matrix/size sweep.
16desc_* configs trip trace.overflow	Benign — bounds the 2048-beat debug trace, not the bus-meter counters.
run_characterization.py -o crashes on save	-o paths are relative to host/; use an absolute path or ../../reports/perf.

7.3 Cannot find the board

The USB-UART re-enumerates on replug. Use --port auto (SCRATCH magic 0xC0FFEE5A) or a stable /dev/serial/by-id/... path — never a hardcoded /dev/ttyUSB*. Confirm alive with dump_status.py → BUILD_ID = 0x5354_5243.

7.4 Not simulatable

The Vivado MCDMA IP is VHDL — Verilator cannot simulate it, so MCDMA validation is FPGA-only. The full iDMA end-to-end (desc64 + backend) is not built (width mismatch); its two halves are characterized separately for the area comparison.

7.5 Board fit vs sim geometry

NUM_CHANNELS is reduced 8 → 4 for sim (Artix BRAM); the as-built characterization bitstream is 8-channel with monitors on (the perf build). Timing closes at +0.031 ns WNS after trimming monitors to perf-only.

7.6 Which README to trust

The dated `flows-stream-bridge/README.md` (“Phase 1”, 2026-04) describes an earlier 5-slave `axil_decode_5s / axil2apb` decoder. The **current** design uses the generated `bridge_stream_char_axil`; trust `PORT_MAP.md` and `host/ADDRESS_MAP.md` (July 2026), and the regmap generator, over that README.